



Universidad de los Llanos

WATER TANK

Proyecto de Tecnologías avanzadas

4913 - 5002 - 5018 - 5022 - 5043



Agenda

01

Problema abordado por el proyecto.

02

Solución propuesta y alcance actual del proyecto.

03

Casos de uso del proyecto

04

Caso de uso implementado y razón de la selección

05

Diagrama de clases UML

06

Diagrama de arquitectura

07

Explicación de los conceptos de POO utilizados

08

Explicación de los conceptos de GRASP utilizados

Agenda

01

Explicación de los
conceptos de SOLID
utilizados

02

Enfoque de persistencia
elegido y justificación para
el proyecto

03

Componentes de
hardware utilizados

07

Evidencias del repositorio

09

Diagrama de clases UML

10

Diagrama de arquitectura

11

Explicación de los conceptos de
POO utilizados

12

Explicación de los conceptos de
GRASP utilizados



WATER TANK Problema abordado

El mantenimiento de **ecosistemas acuáticos cerrados** presenta desafíos críticos debido a la **volatilidad** de sus **parámetros físicos**. La falta de un **sistema automatizado** genera una **dependencia de la supervisión manual**, la cual es propensa a errores de precisión y omisiones temporales. Esto deriva en **riesgos** como el desequilibrio térmico, la evaporación excesiva que altera la salinidad/química del agua y posibles desbordamientos, **comprometiendo** la **vida biológica** y la **integridad del equipo electrónico**.





Solución Propuesta y Alcance

Solución

- Sistema embebido con Arduino y sensores de nivel, temperatura y fuga
- Arquitectura OO con patrones GRASP para alta cohesión y bajo acoplamiento y con patrones SOLID implementados
- Control automático de bomba de agua ante niveles críticos
- Alarma sonora ante condiciones de emergencia
- Consola de comandos para operación manual y monitoreo

Alcance Actual

- Capa de Dominio WaterTank, Sensor, Pump, Alarm, Event
 - Capa de Servicio AuthService, EventHandler, CommandHandler
 - Interfaz Consola ConsoleInterface con parseCommand
 - Persistencia InMemoryUserRepository + IUserRepository
 - Hardware físico Arduino + sensor ultrasónico implementado
 - Diseño UML Diagrama de clases con GRASP y SOLID aplicado
- 

CASOS DE USO

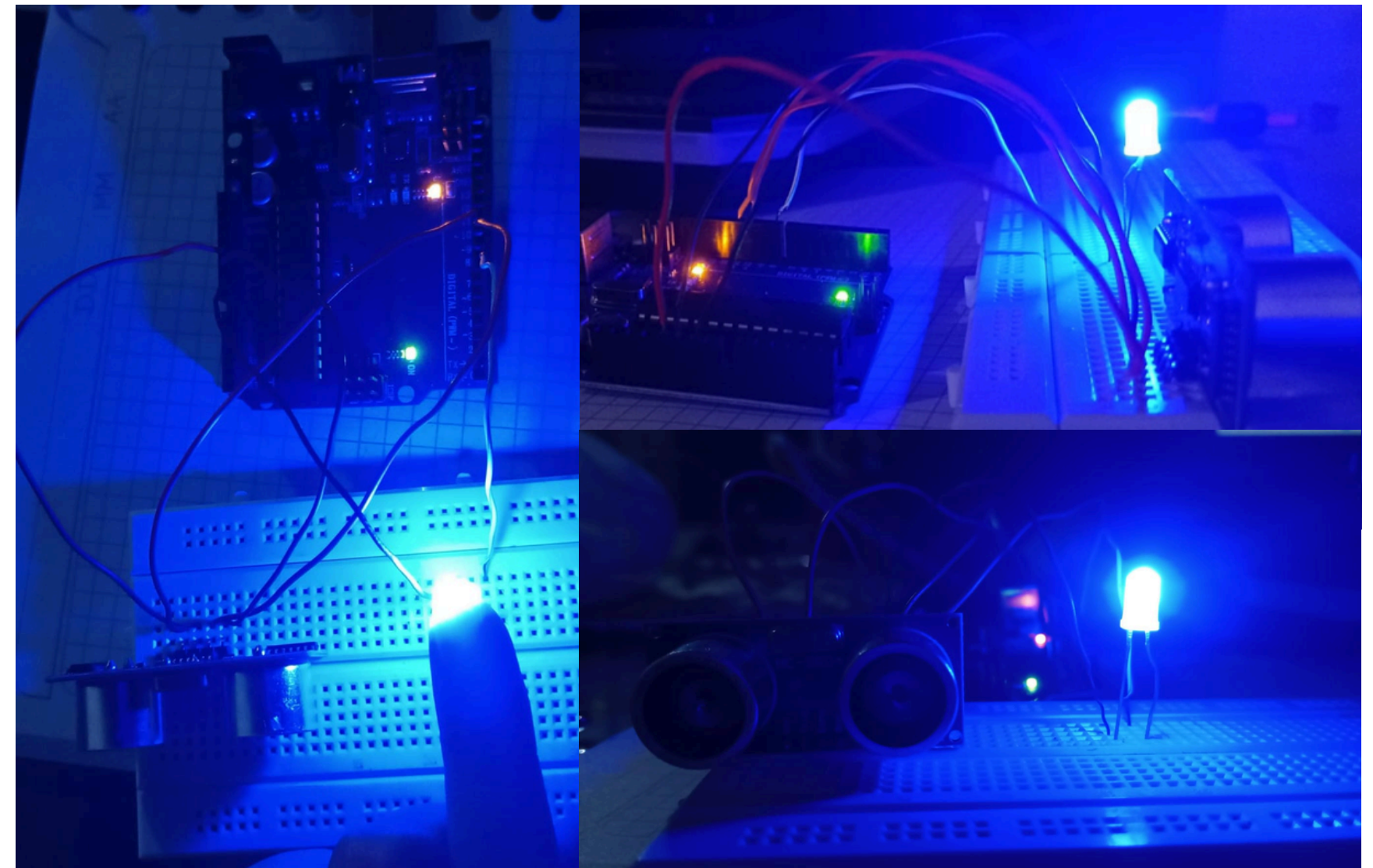
CASO DE USO	SENSOR (ES)	CONDICIÓN DE ACTIVACIÓN	REACCIÓN ESPERADA	DATOS QUE PERSISTAN	IMPLEMENTADO?
Monitoreo de Nivel	Ultrasónico HC-SR04	Nivel < 20% (umbral mínimo). Nivel > 80% (umbral máximo).	Activar rele para encender la Mini-bomba de agua + Mostrar alerta en consola.	Nivel (%), estado bomba (ON/OFF), timestamp de cada evento	SÍ
Alerta de Temperatura	Sensor DS18B20	Temperatura > 35°C (sobrecalentamiento o crítico).	Encender LED rojo + activar alarma sonora + mostrar alerta en consola.	Temperatura (°C), timestamp de alerta, duración de condición anómala.	NO
Vaciado de Emergencia	Comando en Consola	Comando 'DRENAR' del usuario o botón físico presionado.	Abrir válvula de drenaje + detener bomba + confirmar acción en consola.	Acción de usuario, timestamp de inicio, nivel al momento del drenaje.	NO

CASO DE USO IMPLEMENTADO

A través de una **placa microcontroladora**, el **sensor ultrasónico de distancia (cilindros plateados emisor/receptor)** genera una "**lectura bruta**" midiendo el **espacio vacío** en el tanque.

Esta medición en centímetros se transmite como un **flujo de texto a través del cable USB hacia la aplicación en Java**, donde se convierte en un valor numérico que alimenta directamente al objeto **WaterSensor**.

Es gracias a esta **captura física** que el **WaterLevelManager** recibe la información real necesaria para calcular el **porcentaje de llenado** y evaluar si el nivel del agua ha cruzado los límites permitidos para activar la bomba.



CASO DE USO IMPLEMENTADO

El HC-SR04 mide distancia usando ondas de sonido, igual que un murciélago.

Principio básico:

- 1.El pin TRIG emite un pulso de sonido (ultrasonido) hacia el agua
- 2.El sonido rebota en la superficie del agua y regresa
- 3.El pin ECHO recibe el rebote
- 4.El Arduino mide cuánto tiempo tardó en regresar
- 5.Con esa duración calcula la distancia: $\text{distancia} = (\text{tiempo} \times \text{velocidad del sonido}) / 2$

HC-SR04 emite pulso → rebota en agua → Arduino mide tiempo → calcula distancia → convierte a % → decide si activa bomba

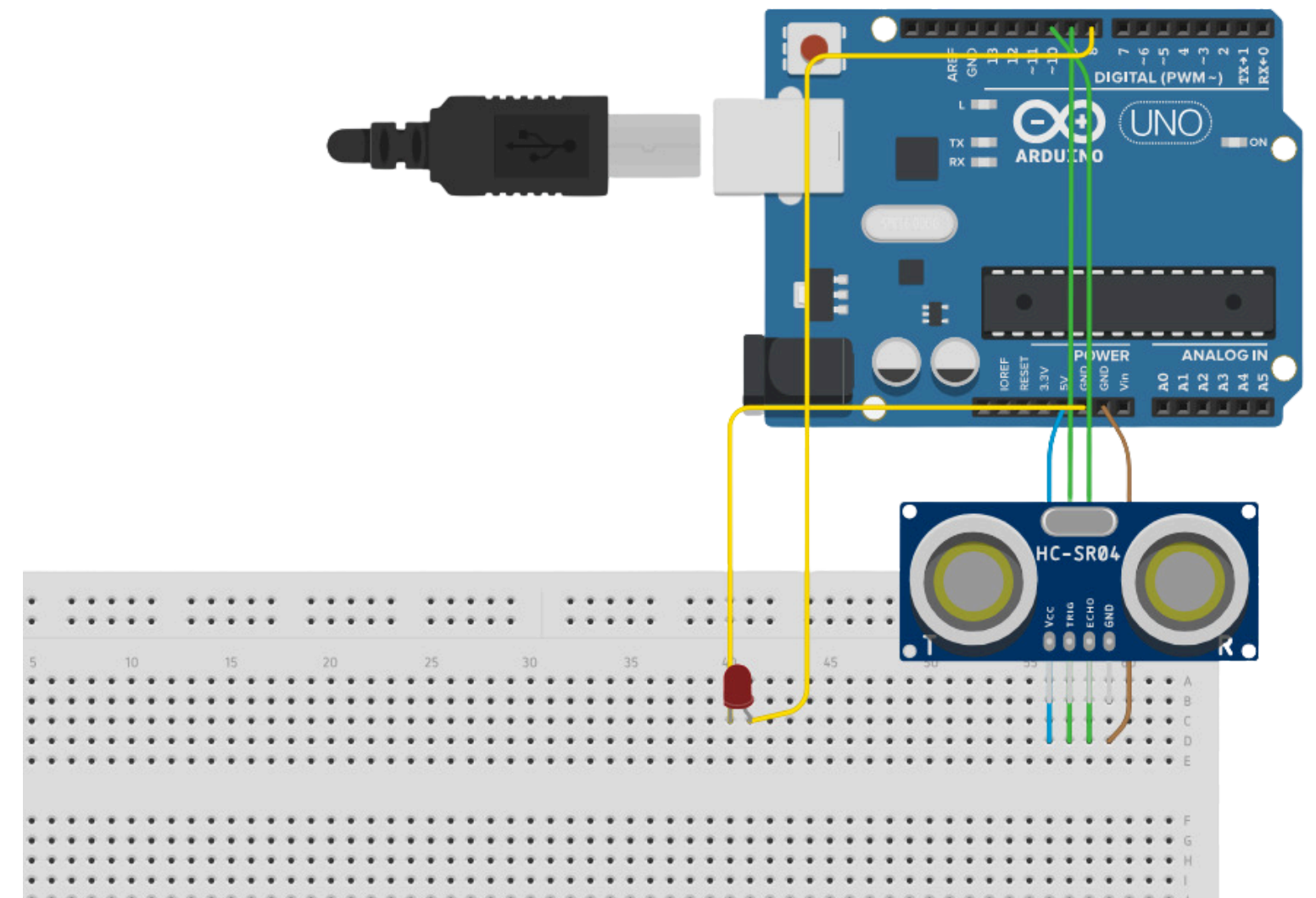


DIAGRAMA DE CLASE UML

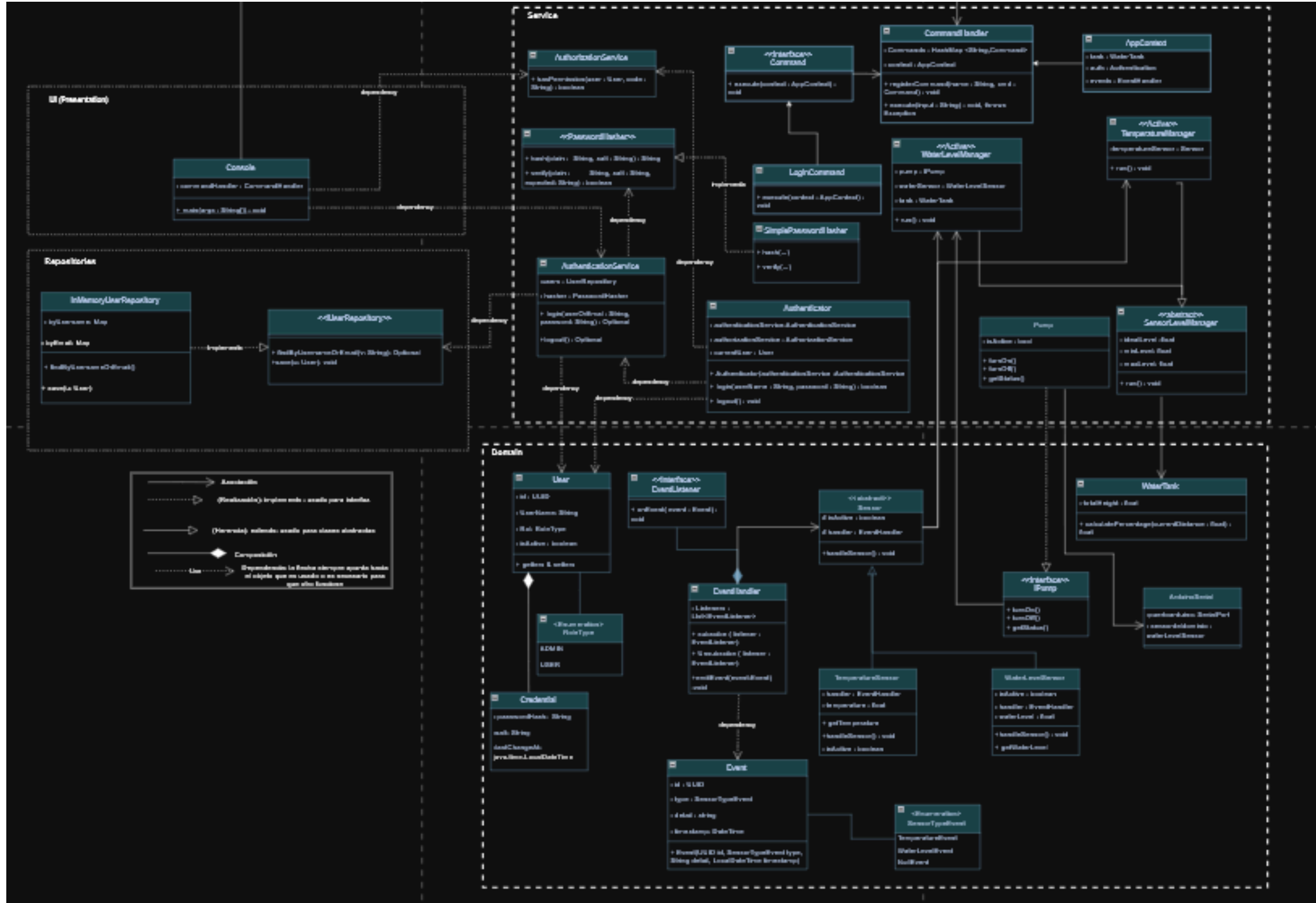
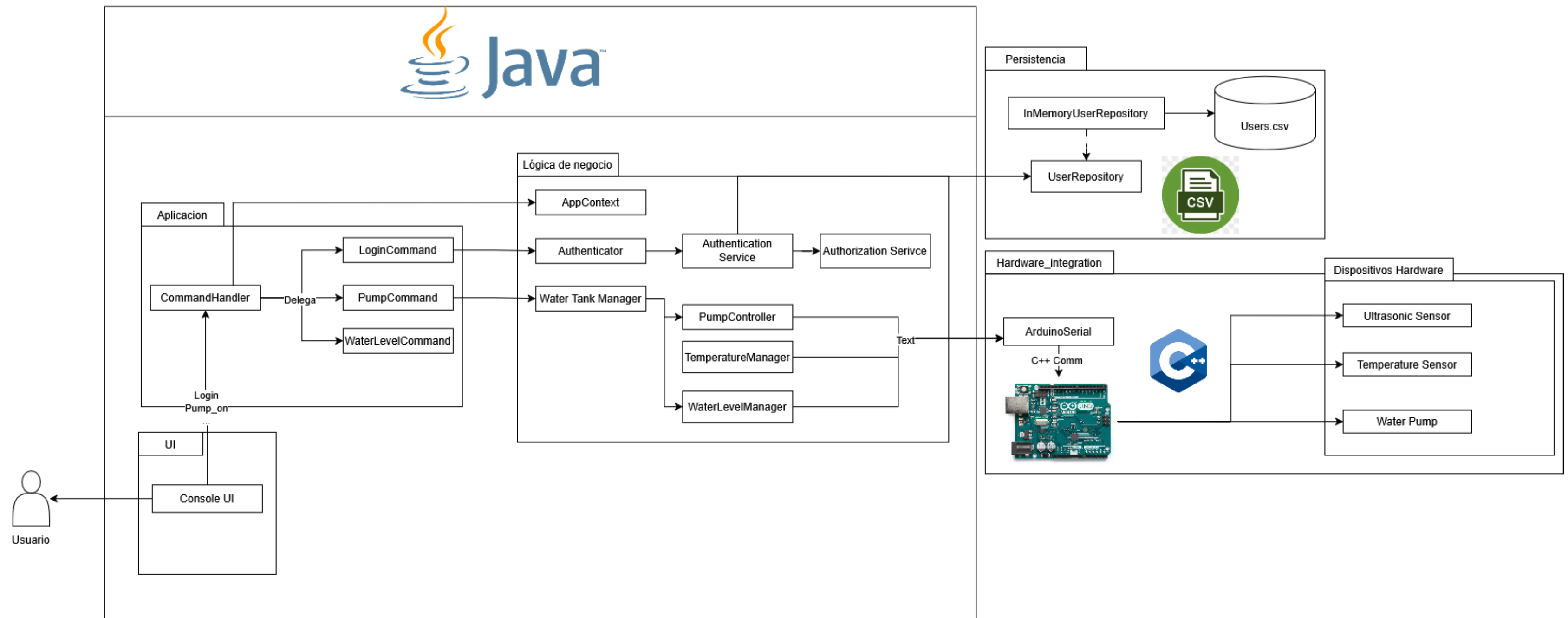


DIAGRAMA DE ARQUITECTURA



Decisiones POO en el proyecto

1

Abstracción: Se visualiza en las clases marcadas como <<abstract>> (Sensor, SensorLevelManager) y las interfaces (IPump, IUserRepository, Command). Estas definen el "qué" debe hacer un componente sin obligar a definir el "cómo" en ese nivel.

2

Encapsulamiento: Se aplica rigurosamente en todas tus clases (como User, WaterTank, Credential) mediante el uso de atributos privados (-) y métodos públicos (+) para el acceso controlado a los datos.

3

Polimorfismo: Se aplica mediante las interfaces. Por ejemplo, el CommandHandler puede ejecutar cualquier objeto que implemente Command (como LoginCommand) sin conocer su lógica interna, tratándolos a todos por igual.

4

Herencia (extends): Se observa en la relación de línea sólida con triángulo vacío entre WaterLevelManager y SensorLevelManager, y entre los sensores específicos (TemperatureSensor) y la clase abstracta Sensor. Los hijos heredan atributos como isActive y handler.



Decisiones GRASP en el proyecto

GRASP

Variaciones Protegidas

Problema

WaterTank dependía directamente de Pump, Sensor y Alarm concretos. Cualquier cambio de hardware rompía el sistema.

Solución

Se crearon interfaces ISensor, IPump e IAlarm. WaterTank solo conoce las interfaces. Los sensores y actuadores físicos se inyectan.

Resultado

Añadir SolarPump o un nuevo sensor no requiere modificar WaterTank.

GRASP

Experto en Información

Problema

SystemController calculaba límites de seguridad con datos del tanque (currentLevel, config), violando quién posee la información.

Solución

Se creó SensorLevelManager (abstracta) con los atributos minLevel, maxLevel e idealLevel, y WaterLevelManager los evalúa directamente teniendo ya inyectados pump: IPump y waterSensor: WaterLevelSensor. Cada clase opera sobre sus propios datos.

Resultado

La lógica de negocio está donde viven los datos. Menos acoplamiento entre capas.

GRASP

Alta Cohesión

Problema

SystemController mezclaba coordinación de comandos, verificación de seguridad y actualización de display en una sola clase.

Solución

Se extrajeron responsabilidades hacia clases dedicadas: WaterLevelManager y TemperatureManager manejan la lógica de cada sensor, CommandHandler gestiona los comandos, EventHandler procesa los eventos, y ConsoleInterface se encarga del display (displayStatus).

Resultado

Cada clase tiene una sola razón de cambio. El God Object fue eliminado completamente del diseño.

Decisiones SOLID en el proyecto

Single Responsibility

Antes: SystemController mezclaba seguridad, display y comandos.

Después: Las responsabilidades se distribuyeron en clases dedicadas: CommandHandler maneja comandos, EventHandler procesa eventos del sistema, WaterLevelManager y TemperatureManager gestionan la lógica de cada sensor, y ConsoleInterface actualiza el display. Cada clase tiene una única responsabilidad.

Open / Closed

Antes: Para agregar un nuevo sensor había que modificar WaterTank y SystemController.

Después: ISensor permite agregar PressureSensor sin modificar ninguna clase existente. Abierto para extensión, cerrado para modificación.

Dependency Inversion

Antes: WaterTank instanciaba directamente Pump, Sensor y Alarm con 'new'.

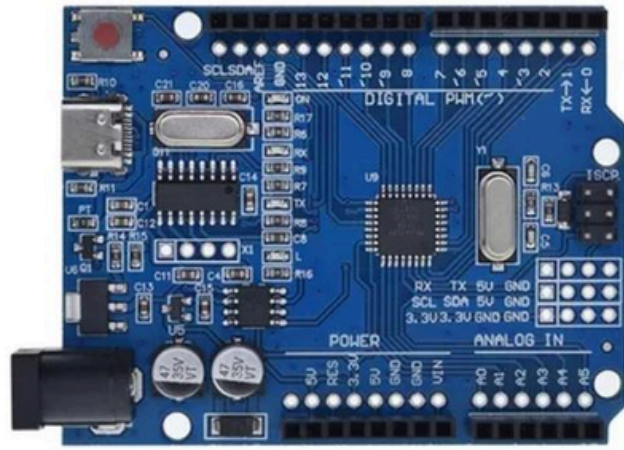
Después: WaterTank depende de IPump, ISensor e IAlarm. Las implementaciones concretas se inyectan desde ApplicationContext, no se crean internamente.

ENFOQUE DE PERSISTENCIA ELEGIDO Y JUSTIFICACIÓN DE SU IDONEIDAD PARA LA ETAPA DEL PROYECTO

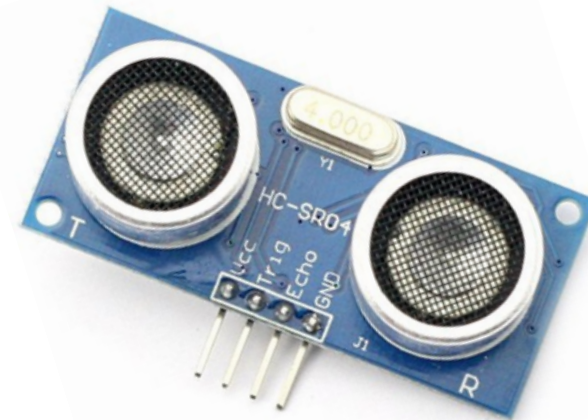
Para este hito del proyecto, se ha implementado un mecanismo de persistencia basado en **archivos de texto plano (logs)** gestionado por la aplicación en Java, no en el microcontrolador. Esta decisión arquitectónica se alinea con el principio de Bajo Acoplamiento y permite un desarrollo ágil del prototipo.

Al **evitar** la configuración inicial de un motor de bases de datos relacional (como MySQL o PostgreSQL), el equipo pudo enfocarse en la **integración del hardware (Arduino)** y la **lógica de negocio (Managers)**. El patrón **Repository** garantiza que, en un futuro, el almacenamiento en **archivo .txt pueda ser sustituido por una base de datos real** sin alterar el código del Dominio ni los Servicios.

Componentes de Hardware utilizados



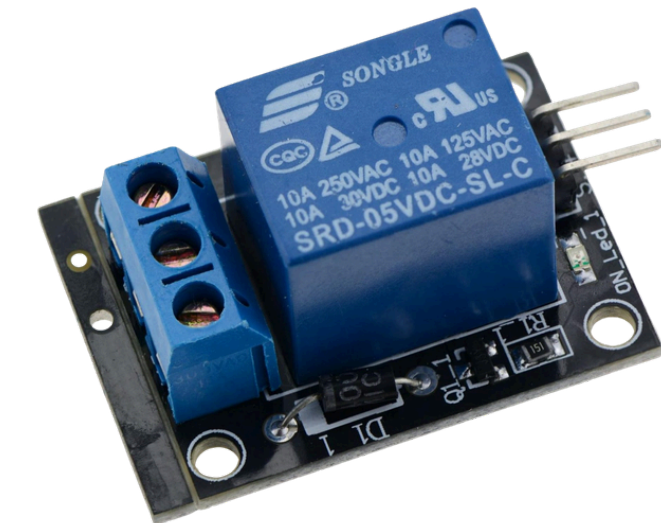
Arduino UNO R3
Microcontrolador ATmega328p



Sensor de Distancia de Ultrasonido HC-SR04



Sensor De Temperatura Ds18b20



Módulo relé electromecánico de 1 canal (5V)



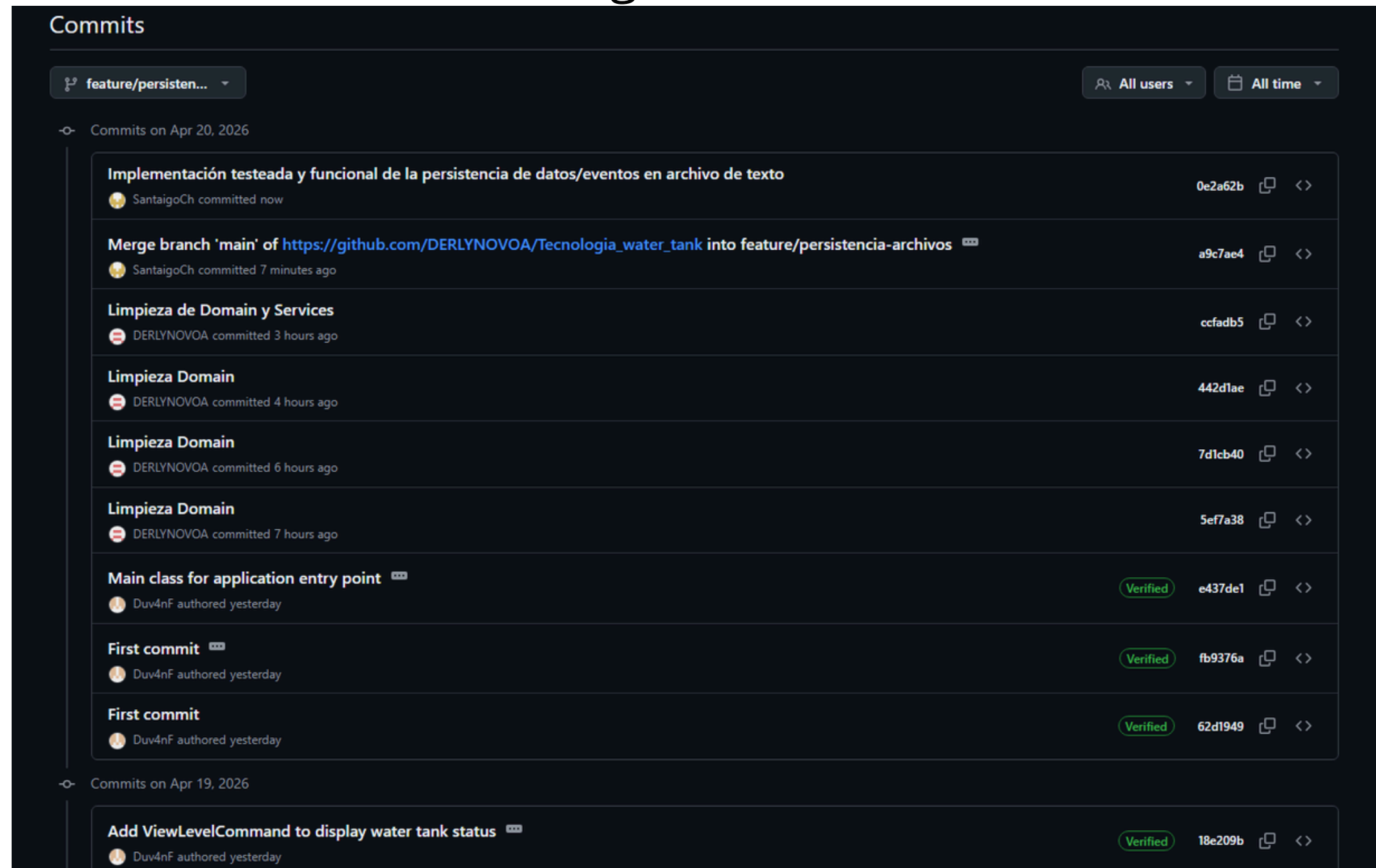
Calentador de agua



Bomba de agua 5v

EVIDENCIA REPOSITORIO

GIT HUB ramas, confirmaciones y contribución de todos los miembros del equipo.
Santiago Chavarro



EVIDENCIA REPOSITORIO

GIT HUB ramas, confirmaciones y contribución de todos los miembros del equipo.

Duvan Baquero

Commits

History for `Tecnologia_water_tank` / Backend / src / main / java on `main`

All users All time

Commits on Apr 20, 2026

Main class for application entry point

Duv4nF authored 1 hour ago

Verified e437de1

First commit

Duv4nF authored 2 hours ago

Verified fb9376a

First commit

Duv4nF authored 2 hours ago

Verified 62d1949

Commits on Apr 19, 2026

Add ViewLevelCommand to display water tank status

Duv4nF authored 2 hours ago

Verified 18e209b

EVIDENCIA REPOSITORIO

GIT HUB ramas, confirmaciones y contribución de todos los miembros del equipo.

Astrid Martinez

Default					
Branch	Updated	Check status	Behind Ahead	Pull request	
main 	 49 minutes ago		Default		
Your branches					
Branch	Updated	Check status	Behind Ahead	Pull request	
Domain_development_and_cleanup 	 53 minutes ago		0 0		
Active branches					
Branch	Updated	Check status	Behind Ahead	Pull request	
Domain_development_and_cleanup 	 53 minutes ago		0 0		

EVIDENCIA REPOSITORIO

GIT HUB ramas, confirmaciones y contribución de todos los miembros del equipo.

Derly Novoa

Commits

main

DERLYNOVOA

All time

Commits on Apr 20, 2026

Limpieza de Domain y Services

DERLYNOVOA committed 4 hours ago

ccfadb5

<>

Limpieza Domain

DERLYNOVOA committed 6 hours ago

442d1ae

<>

Limpieza Domain

DERLYNOVOA committed 8 hours ago

7d1cb40

<>

Limpieza Domain

DERLYNOVOA committed 9 hours ago

5ef7a38

<>

Commits on Apr 12, 2026

Add README.md file

DERLYNOVOA authored last week

Verified

a13ab9c

<>

EVIDENCIA REPOSITORIO

GIT HUB ramas, confirmaciones y contribución de todos los miembros del equipo.

Juan Romero

The screenshot displays a GitHub commit interface. At the top, the commit is identified by the hash `9db114b` and the user `JuDROM`, who committed it 4 days ago. The commit message is `first commit`. Below this, the commit is linked to the `main` branch. The interface shows that 44 files were changed, resulting in 957 lines added and 0 lines removed. A file explorer on the left lists the project structure: `Backend/src/main/java`, `ArduinoComm` (containing `ArduinoSerial.java`), and `Commands` (containing `Command.java` and `CommandHandler.java`). The main area displays the diff for `Backend/src/main/java/ArduinoComm/ArduinoSerial.java`, showing four lines of new code: `package ArduinoComm;`, `public class ArduinoSerial {`, and its closing brace. A search bar at the top right allows searching within the code.

Commit `9db114b` [Browse files](#)

`JuDROM` committed 4 days ago

`first commit`

`main` 1 parent `a13ab9c` commit `9db114b`

Filter files... 44 files changed +957 -0 lines changed Search within code

Backend/src/main/java ArduinoComm ArduinoSerial.java +4

```
@@ -0,0 +1,4 @@
1 + package ArduinoComm;
2 +
3 + public class ArduinoSerial {
4 + }
```


APLICACIÓN DE CONSOLA

Al iniciar, se encarga de **instanciar y "conectar" los componentes de hardware** con la **lógica de negocio, inyectando las dependencias** para que los **Managers** sepan qué sensores y bombas controlar. Además, sirve como una capa de aislamiento; recibe las instrucciones del usuario y utiliza el **patrón Command** para delegar la ejecución al **dominio**, permitiendo visualizar los registros y alertas en pantalla sin que la interfaz gráfica se mezcle con las reglas matemáticas del sistema.

LECTURA DEL SENSOR

A. El proceso inicia cuando el **sensor ultrasónico HC-SR04** mide la **distancia física** y Arduino la envía a Java. Allí, la clase **WaterTank** actúa como "**experto en información**" **traduciendo** el dato a un **porcentaje de llenado**. Inmediatamente, se emite un **evento (patrón Observer)** hacia el **WaterLevelManager**, quien **verifica** si el nivel cruzó sus límites críticos.

B. De forma simultánea, el **sensor físico DS18B20** captura la **temperatura** y **actualiza** directamente el **estado del objeto TemperatureSensor**. Aplicando el mismo patrón de eventos, este sensor "despierta" al **TemperatureManager**, el cual compara la nueva lectura con sus umbrales predefinidos (ideal, mínimo y máximo) para decidir si el sistema requiere intervenir.

REACCIÓN DEL SISTEMA

La **reacción** es **gestionada por el WaterLevelManager**, que atrapa el **evento del sensor** y compara el **porcentaje actual** con sus **reglas de negocio**. Si detecta que el agua cayó por debajo del límite **mínimo** (por ejemplo, 20%), ejecuta el **método turnOn()**, enviando una señal al hardware para **encender un LED o activar el relé** de la bomba real y comenzar el llenado.

A medida que el tanque se llena, el sistema sigue evaluando las lecturas hasta que el nivel alcanza el límite **máximo** configurado (por ejemplo, 80%); en ese momento, el **manager emite la orden turnOff()**, **apagando el LED y deteniendo el proceso** de forma totalmente automatizada y segura.

RESULTADO DE LA PERSISTENCIA

La persistencia se localiza en la capa de **Repositories**, específicamente a través de la clase **InMemoryUserRepository**. Los datos se almacenan de forma estructurada utilizando colecciones de Java, como el **atributo Map**, que permite organizar la información (usuarios, credenciales o estados) mediante **claves únicas** para una **recuperación inmediata**.

El proceso técnico ocurre cuando la **capa de Service interactúa con la interfaz IUserRepository** para **registrar cambios** o consultar el **historial**. Al utilizar este patrón, el sistema asegura que el estado del tanque y los eventos del sistema no se pierdan durante la ejecución, permitiendo que el administrador audite cada alerta disparada por los sensores directamente desde los registros almacenados.

FIN.

**GRACIAS POR
SU ATENCION**



**PROFEME CONFORMO
CON UN 5.0**